

# Dynamically Maintaining Conversation History

## 1. Today's Most Important Pattern

The most important coding pattern is:

```
messages.append(  
    HumanMessage(content=user_input)  
)
```

```
response = llm.invoke(messages)
```

```
messages.append(response)
```

This is the core logic for maintaining conversation history dynamically.

---

## 2. Core Flow

Add Human Message



Send History to Model



Get AI Response



Add AI Response



Repeat

This cycle is repeated for every new user message.

---

### **3. Why Is This Pattern Important?**

For every conversation turn:

User enters message



Create HumanMessage



Add it to history



Send complete history to model



Receive AI response



Add AI response to history

So conversation history keeps growing dynamically.

---

## 4. Program 1 — First Dynamic Conversation History Program

Start with an empty list.

Then:

1. Add the human message.
2. Send the complete history to the model.
3. Get the AI response.
4. Add the AI response back to the list.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
# Empty conversation history
```

```
messages = []
```

```
# User message
```

```
user_input = "My name is Durga."
```

```
# Add user message to history
```

```
messages.append(
```

```
    HumanMessage(
```

```
        content=user_input
```

```
    )
```

```
)
```

```
# Send complete history to model
```

```
response = llm.invoke(
```

```
    messages
```

```
)
```

```
print("AI:", response.content)
```

```
# Add AI response to history  
messages.append(  
    response  
)
```

```
print("\nConversation History:")  
print(messages)
```

---

## 5. Program Flow

```
messages = []
```



```
Add HumanMessage
```



```
llm.invoke(messages)
```



Receive AIMessage



Add AIMessage



Updated History

---

## **6. Very Important Point**

The object returned by:

```
response = llm.invoke(messages)
```

is already an AI response message.

So we can directly write:

```
messages.append(response)
```

We do not need to manually create another AI message.

---

## **7. History After One Conversation Turn**

Suppose the user says:

My name is Durga.

The model replies:

Nice to meet you, Durga.

Then the history conceptually contains:

messages

```
|
|— HumanMessage("My name is Durga.")
|
|— AIMessage("Nice to meet you, Durga.")
```

Now this history can be used in the next model call.

---

## 8. Interactive Chatbot with Exit

Now we can repeat the same logic using a loop.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
llm = ChatOpenAI(
    model="gpt-4o-mini",
    api_key=os.getenv("OPENAI_API_KEY")
)
```

```
messages = []
```

```
while True:
```

```
    user_input = input(  
        "You: "  
    )
```

```
    # Stop chatbot
```

```
    if user_input.lower() == "exit":  
        print("Chat ended.")  
        break
```

```
    # Add user message
```

```
    messages.append(  
        HumanMessage(  
            content=user_input
```

```
)  
  
)  
  
# Send complete history  
response = llm.invoke(  
    messages  
)  
  
# Display AI response  
print(  
    "AI:",  
    response.content  
)  
  
# Add AI response  
messages.append(  
    response  
)  
  
print(  

```

```
"\nNumber of messages:",  
len(messages)  
)
```

---

## 9. How the Chatbot Works

Every loop iteration performs:

Read User Input



Append HumanMessage



Send Complete History



Get AI Response



Print AI Response



Append AIMessage



Repeat

---

## 10. Why Do We Use a Loop?

Without a loop:

One Question



One Response



Program Ends

With a loop:

Question 1



Answer 1



Question 2



Answer 2



Question 3



Answer 3



...

This creates a multi-turn chatbot.

---

## **11. Why Do We Use exit?**

We check:

```
if user_input.lower() == "exit":
```

If the user types:

exit

the program executes:

break

and the chat ends.

---

## **12. Why user\_input.lower()?**

Suppose the user enters:

EXIT

Exit

exit

After:

```
user_input.lower()
```

all become:

exit

So the program can handle different letter cases.

---

### **13. Number of Messages**

We can check the number of stored messages using:

`len(messages)`

After one full conversation turn:

HumanMessage

AIMessage

Number of messages:

2

After two full turns:

HumanMessage

AIMessage

HumanMessage

AIMessage

Number of messages:

4

So normally:

1 complete turn



2 messages

---

## 14. Program — Print Complete Conversation History

We can also print the complete conversation history.

```
import os
```

```
from langchain_openai import ChatOpenAI
```

```
from langchain_core.messages import HumanMessage
```

```
llm = ChatOpenAI(
```

```
    model="gpt-4o-mini",
```

```
    api_key=os.getenv("OPENAI_API_KEY")
```

```
)
```

```
messages = []
```

```
while True:
```

```
    user_input = input(  
        "\nYou: "  
    )
```

```
    if user_input.lower() == "exit":  
        print("Chat ended.")  
        break
```

```
    messages.append(  
        HumanMessage(  
            content=user_input  
        )  
    )
```

```
    response = llm.invoke(  
        messages  
    )
```

```
print(  
    "AI:",  
    response.content  
)
```

```
messages.append(  
    response  
)
```

```
# -----  
# PRINT CURRENT HISTORY  
# -----
```

```
print("\nCurrent History:")  
print("-----")
```

```
for message in messages:
```

```
    print(  
        "
```

```
    type(message).__name__,  
    ":",  
    message.content  
)
```

---

## 15. Understanding the History Loop

This loop:

for message in messages:

visits every stored message.

This:

```
type(message).__name__
```

gives the message type.

Example:

HumanMessage

AIMessage

And:

```
message.content
```

gives the actual text of the message.

---

## 16. Example History Output

Suppose the conversation is:

You: My name is Durga.

AI: Nice to meet you, Durga.

You: What is my name?

AI: Your name is Durga.

The history may look like:

Current History:

-----

HumanMessage : My name is Durga.

AIMessage : Nice to meet you, Durga.

HumanMessage : What is my name?

AIMessage : Your name is Durga.

---

## 17. Memory-Like Behaviour

We can now see memory-like behaviour being created through the list.

messages



Stores previous conversation



History sent again to model



Model gets previous context



Memory-like behaviour

The important point is:

Application maintains the history.

---

## 18. Why Learn Manual History First?

Later, we may see something like:

`RunnableWithMessageHistory`

Before learning that, we should understand the basic problem it is solving.

Without understanding:

`messages.append(...)`

we may think conversation memory is magic.

But now we know history must be:

Stored



Updated



Retrieved



Passed

---

## 19. Manual History Logic

HISTORY

|

|— Store

|

|— Update

|

|— Retrieve

|

└— Pass to Model

This is the foundation for more advanced conversation-history mechanisms.

---

## 20. Limitation 1 — In-Memory History Only

Our current program uses:

```
messages = []
```

This list exists only while the Python program is running.

What happens when the program stops?

Program Stops



Python Process Ends



messages List Disappears

When we run the program again:

```
messages = []
```

starts empty again.

So the current history is:

In-Memory History

Persistent history is a later topic.

---

## 21. Limitation 2 — No Separate Conversation History

Our current program has only:

```
messages = []
```

That means only one conversation history.

Suppose:

Durga is chatting

Ravi is chatting

Suresh is chatting

Should all three users use the same list?

No

They need separate histories.

Concept:

Durga



History 1

Ravi



History 2

Suresh



History 3

This leads us later to concepts such as:

Session ID

Thread ID

Separate History per Session

---

## **22. Why Separate Histories Are Required**

Suppose:

Durga:

My favourite language is Python.

and:

Ravi:

My favourite language is Java.

If both users share the same history:

Durga History

+

Ravi History

their conversation data may get mixed.

So each conversation should maintain its own history.

---

## 23. Limitation 3 — Growing History Problem

Our list keeps growing.

Example:

2 messages

4 messages

10 messages

50 messages

100 messages

...

Eventually this may affect:

Context Window

Token Usage

Cost

Latency

---

## 24. Growing History Flow

More Conversation



More Messages



More Content



More Context Usage



Potentially More Cost



Potentially More Latency

This is why conversation-history management becomes important in larger applications.

---

## 25. Limitation Summary

| Limitation      | Meaning                                             |
|-----------------|-----------------------------------------------------|
| In-memory only  | History disappears when program stops               |
| Single history  | No separate history for different users or sessions |
| Growing history | History keeps increasing over time                  |

---

## 26. Common Mistake 1 — Wrong Order

Wrong:

```
response = llm.invoke(messages)
```

```
messages.append(  
    HumanMessage(  
        content=user_input  
    )  
)
```

Here the latest user question is added after model invocation. So the model does not receive the latest user message in that call.

**Correct**

```
messages.append(  
    HumanMessage(  
        content=user_input  
    )  
)
```

```
response = llm.invoke(  
    messages
```

)

## **Memory Rule**

First Append Human



Then Invoke Model

---

## **27. Common Mistake 2 — Forgetting AI Response**

If we forget:

```
messages.append(response)
```

then AI responses will not be present in future conversation history.

Correct flow:

Append Human



Invoke



Append AI

---

## **28. Why Store AI Responses?**

Conversation history normally contains both sides.

Human Message

+

AI Message

If we store only user messages:

Human

Human

Human

then the model does not receive the complete previous dialogue.

---

### **29. Common Mistake 3 — Creating messages = [] Inside the Loop**

Wrong:

while True:

```
    messages = []
```

Ayyooo!

Every loop iteration creates a new empty list.

So previous history is lost.

---

### **30. Correct Placement**

Correct:

```
messages = []
```

```
while True:
```

```
    ...
```

Create the list before the loop.

Very very important.

---

### **31. Why Should the List Be Outside the Loop?**

Wrong flow:

Loop Starts



Create Empty List



Conversation



Next Iteration



Create Empty List Again

Previous history is lost.

Correct flow:

Create One List



Start Loop



Keep Updating Same List



Conversation Continues

---

## **32. Common Mistake 4 — Sending Only user\_input**

Inside a conversation chatbot, if we write:

```
llm.invoke(user_input)
```

we are sending only the latest user input.

For conversation history, the important call is:

```
llm.invoke(messages)
```

### **Difference**

```
llm.invoke(user_input)
```



Latest input only

versus:

```
llm.invoke(messages)
```



Complete maintained history

---

### **33. Common Mistake 5 — Appending Only Strings**

Avoid:

```
messages.append(  
    user_input  
)
```

For our message-based example, use:

```
messages.append(  
    HumanMessage(  
        content=user_input  
    )  
)
```

This makes the message role clear.

---

### **34. Why Use HumanMessage?**

Using:

```
HumanMessage(...)
```

clearly represents:

Message from Human

And the model response represents:

Message from AI

So history contains clear conversation roles.

---

### **35. Correct Dynamic Conversation Pattern**

```
messages = []
```

```
while True:
```

```
    user_input = input("You: ")
```

```
    messages.append(
        HumanMessage(
            content=user_input
        )
    )
```

```
    response = llm.invoke(
```

```
    messages
)

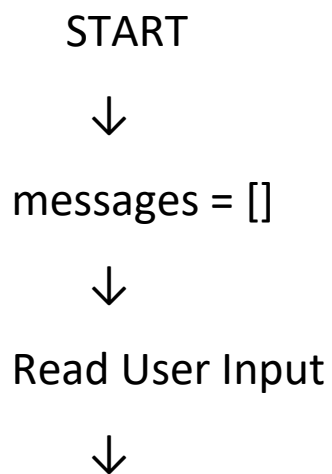
print(
    "AI:",
    response.content
)

messages.append(
    response
)
```

This is the core concept of this topic.

---

## 36. Master Flow



Create HumanMessage



Append HumanMessage



Send Complete History



llm.invoke()



Receive AIMessage



Display AI Response



Append AIMessage



Updated History



Next User Input



REPEAT



### 37. Important Key Points

1. Conversation history can be maintained using a Python list.
2. User input is added as a HumanMessage.
3. The complete messages list is passed to the model.
4. The model response can be appended directly to the conversation history.
5. A loop converts the same logic into a multi-turn chatbot.
6. `messages = []` should be created before the chat loop.
7. Both human and AI messages should be preserved.
8. The application is responsible for maintaining the history.
9. Current list-based history exists only while the program is running.
10. Different users or sessions need separate conversation histories.
11. Growing history can affect context window, token usage, cost, and latency.

---

### 38. MCQ 1

Where should we create this list?

```
messages = []
```

- A. Inside every loop iteration
- B. Before the chat loop
- C. After the program ends
- D. Inside response.content

**Answer: B**

### **Conclusion**

Create the history once before the loop so previous messages remain available.

---

### **39. MCQ 2**

Which line sends previous conversation history to the model?

**A**

`llm.invoke(user_input)`

**B**

`llm.invoke(messages)`

**C**

`print(messages)`

**D**

`messages.clear()`

**Answer: B**

### **Conclusion**

llm.invoke(messages) sends the complete currently maintained conversation history.

---

#### 40. MCQ 3

Why do we execute:

`messages.append(response)`

- A. To retrain the model
- B. To store the AI response in conversation history
- C. To create an API key
- D. To clear conversation history

**Answer: B**

#### Conclusion

The AI response becomes part of the history for future conversation turns.

---

#### 41. Final Revision Diagram

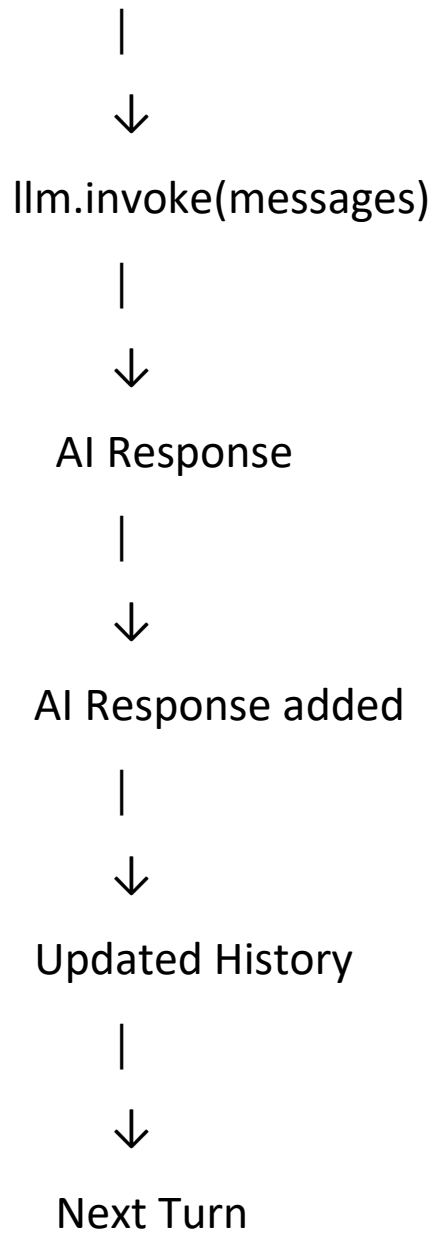
CONVERSATION HISTORY

messages

|



HumanMessage added



---

## 42. Easy Memory Formula

HUMAN



APPEND



INVOKE



AI



APPEND



REPEAT

Or simply:

Append Human



Invoke Model



Append AI



Repeat

---

### 43. Manual Conversation History

messages = []



History maintained by application



History keeps growing



Complete history sent to model



Model gets previous context

---

## **44. Why Manual History Matters**

Before learning higher-level tools such as:

`RunnableWithMessageHistory`

we should understand the basic logic:

Store



Update



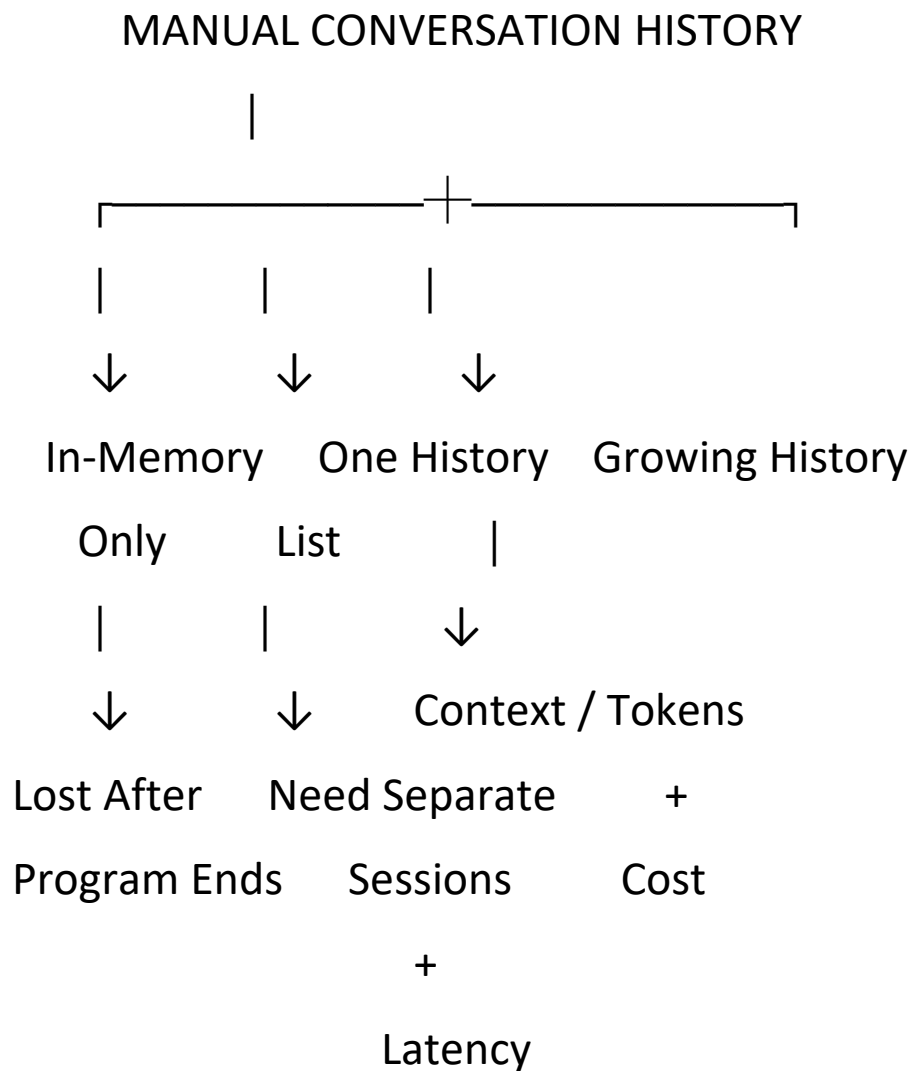
Retrieve



Pass

Once this concept is clear, advanced conversation-history tools become easier to understand.

## 45. Master Limitations Diagram



## 46. One-Line Conclusion

**A conversation chatbot works by continuously adding the user's message and the AI's response to a history and sending that updated history back to the model.**

---

## **47. Final Memory Line**

User Message



HumanMessage



Append to History



Send History to Model



AI Response



Append to History



Repeat

The most important coding pattern is:

```
messages.append(
```

```
    HumanMessage(content=user_input)
```

)

```
response = llm.invoke(messages)
```

```
messages.append(response)
```